

CONFIDENTIAL

SECURITY ASSESSMENT

Security Assessment Report

InternalAPI v1.0 (Node.js/Express)



Assessment Date	2026-08-27
Version	1.0
Overall Risk	Critical
Prepared by	SecureReview Inc.

This report is CONFIDENTIAL and intended solely for authorized recipients. It contains sensitive security findings and remediation guidance. Unauthorized disclosure, reproduction, or distribution is strictly prohibited.

Executive Summary

This report presents the findings of a security assessment conducted on InternalAPI v1.0 (Node.js/Express). The assessment focused on source code review and configuration analysis of the Node.js/Express application. A total of 4 security findings were identified, of which 4 remain open and 0 have been remediated.

Overall Risk Rating

CRITICAL

Based on the presence of a Critical SQL Injection vulnerability and multiple High-severity findings, the overall risk to the application is rated Critical. Immediate remediation is required.

Finding Summary

Severity	Open	Remediated	Total
● Critical	1	0	1
● High	2	0	2
● Medium	1	0	1
● Low	0	0	0
Total	4	0	4

Scope and Methodology

Assessment Scope

The assessment covered the complete source code of InternalAPI v1.0, a Node.js/Express web application. The following files were reviewed:

File	Description
src/server.ts	Application entry point, middleware configuration
src/auth.ts	Authentication endpoints (login, token verification)
src/users.ts	User management endpoints, database queries

Methodology

Method	Description
Source Code Review	Manual review of all TypeScript source files for security anti-patterns, dangerous API usage, and logic flaws.
Configuration Review	Analysis of middleware configuration, CORS policy, authentication settings, and secret management practices.

Findings

F-01 — SQL Injection in User Search Endpoint

CRITICAL

OPEN

Location	src/users.ts:27
Description	The GET /users/search endpoint constructs an SQL query by directly interpolating the username query parameter into a string template literal. An attacker can craft input such as <code>\` OR \\'1\'=\'1</code> to bypass filters or <code>\`; DROP TABLE users;--</code> to corrupt or destroy data, all without any authentication.
Impact	Full read/write access to the database. An attacker may ex-filtrate all user records including passwords, modify or delete any row, or drop entire tables. Because the /users/search endpoint requires no authentication token, this attack is available to the public internet.
Rec-ommendation	Replace string interpolation with a parameterized query: <code>db.prepare("SELECT id, username, email, role FROM users WHERE username = ?").all(username)</code> . Never concatenate or interpolate untrusted user input into SQL strings.

F-02 — Hardcoded JWT Secret

HIGH

OPEN

Location	src/auth.ts:5
Description	The JWT signing secret ("super-secret-key-do-not-share-1234") is hardcoded as a string literal in src/auth.ts and duplicated in src/users.ts. Anyone with read access to the repository — including former employees or compromised CI systems — knows the secret in perpetuity.
Impact	An attacker who knows the secret can forge valid JWT tokens for any username and role, including {role: "admin"}, gaining full administrative access to the application without needing a password.
Recommendation	Load the secret exclusively from an environment variable (e.g., process.env.JWT_SECRET) and refuse to start the server if the variable is absent. Rotate the secret immediately, as the current value must be treated as compromised. Use a cryptographically random value of at least 256 bits.

F-03 — Overly Permissive CORS Policy

MEDIUM

OPEN

Location src/server.ts:9–12

Description The CORS middleware sets Access-Control-Allow-Origin, Access-Control-Allow-Methods, and Access-Control-Allow-Headers all to "*". This allows any web page on any origin to make cross-origin requests to the API.

Impact A malicious website can issue authenticated requests on behalf of a logged-in user and read the response. Combined with the hardcoded JWT secret (F-02), cross-site request forgery and data exfiltration scenarios become trivial to exploit.

Recommendation Restrict Access-Control-Allow-Origin to a specific allowlist of trusted origins. Remove the wildcard from Allow-Methods and Allow-Headers, and enumerate only the methods and headers the API actually uses. Consider using the cors npm package with an explicit options object.

F-04 — Debug Endpoint Exposing Environment Variables

HIGH

OPEN

Location	src/server.ts:21–27
Description	The GET /debug endpoint is reachable without any authentication and returns the complete process.env object along with uptime, memory usage, and PID. Environment variables commonly contain database connection strings, API keys, cloud credentials, and the JWT_SECRET value.
Impact	Any unauthenticated attacker who discovers this endpoint obtains all secrets stored in the environment, enabling lateral movement to databases and third-party services. Combined with F-02, obtaining JWT_SECRET via this endpoint allows immediate admin token forgery.
Recommendation	Remove the /debug endpoint entirely from the codebase before any non-development deployment. If internal diagnostics are necessary, protect the endpoint with strong authentication and restrict access by IP allowlist or network policy.

Summary and Conclusion

The security assessment of InternalAPI v1.0 (Node.js/Express) identified 4 vulnerabilities across three source files. The most critical issue is an SQL Injection vulnerability in the user search endpoint (F-01), which allows unauthenticated attackers to read or modify all database records. Equally concerning is the use of a hardcoded JWT secret (F-02) committed to the repository, which enables token forgery for any user account.

Additionally, the application's CORS policy (F-03) is configured to accept requests from any origin, and a debug endpoint (F-04) exposes environment variables without authentication. These findings collectively represent a significant risk to the confidentiality, integrity, and availability of the system.

It is strongly recommended to address all Critical and High findings before the application is deployed or exposed to untrusted networks. The Medium finding should be resolved in the next sprint cycle. Remediated items should be verified through re-testing.

Revision History

Ver- sion	Date	Description	Author
1.0	2026-08-27	Initial report — 4 findings identified	Secur- eReview Inc.
